EXPERIMENT - 7

AIM

To analyze and determine functional dependencies within a denormalized sales database, progressing through normalization stages from First Normal Form (1NF) up to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). The experiment also includes verifying lossless decomposition and dependency preservation using SQL.

1. Database and Denormalized Table Creation

Explanation: We start with a denormalized table storing comprehensive details of sales, customers, staff, and products. This structure inherently suffers from data redundancy, as customer and staff details repeat for multiple items in a single transaction.

```
-- Create database
CREATE DATABASE IF NOT EXISTS RetailSalesDB;
USE RetailSalesDB;

-- Create denormalized table
DROP TABLE IF EXISTS Retail_Sales;
CREATE TABLE Retail_Sales (
    transaction_id INT,
    transaction_date DATE,
    buyer_id INT,
    buyer_name VARCHAR(50),
    agent_id INT,
    agent_name VARCHAR(50),
    item_id INT,
    item_name VARCHAR(50),
    qty INT
);

-- Insert sample records
INSERT INTO Retail_Sales VALUES
(5001, '2026-02-15', 10, 'Rohan', 901, 'Amit', 301, 'Tablet', 1),
(5001, '2026-02-15', 10, 'Rohan', 901, 'Amit', 302, 'Headphones',
2),
(5002, '2026-02-16', 20, 'Kavya', 902, 'Neha', 301, 'Tablet', 1),
(5002, '2026-02-16', 20, 'Kavya', 902, 'Neha', 303, 'Charger',
1),
(5003, '2026-02-17', 10, 'Rohan', 901, 'Amit', 302, 'Headphones',
3),
```

```
(5003, '2026-02-17', 10, 'Rohan', 901, 'Amit', 304, 'Webcam', 1);

-- Display denormalized data
SELECT * FROM Retail_Sales;
```

2. Identification of Functional Dependencies

Explanation: Functional dependencies map out how specific attributes uniquely define other attributes in the dataset.

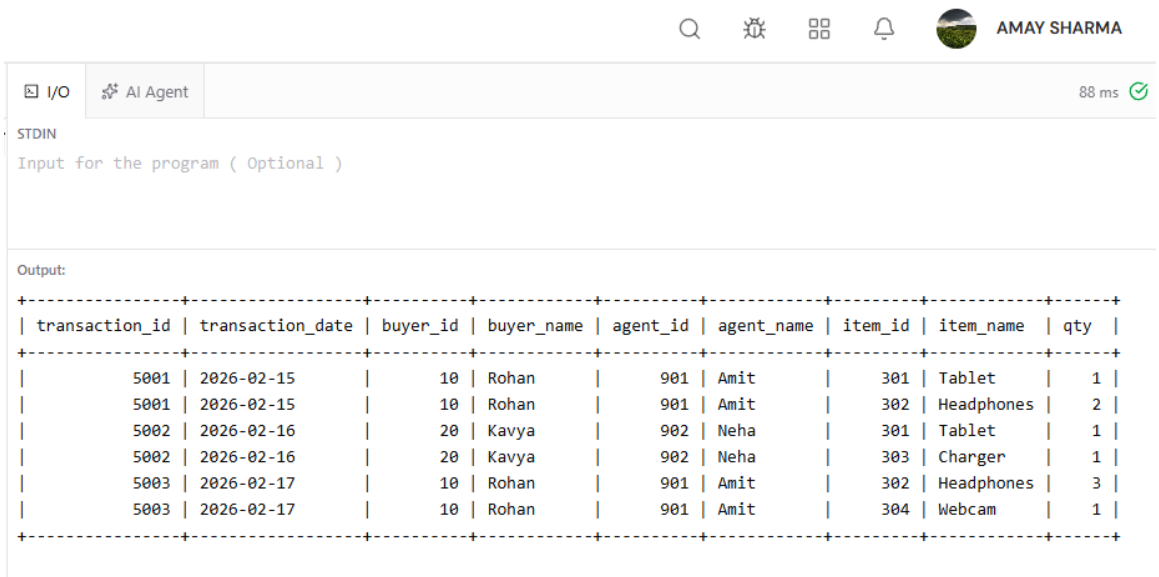
• transaction_id → transaction_date, buyer_id, agent_id (Transaction specifics)
• buyer_id → buyer_name (Buyer specifics)
• agent_id → agent_name (Agent specifics)
• item_id → item_name (Item specifics)
• (transaction_id, item_id) → qty (Quantity for a specific item in a specific transaction)

Candidate Key for the initial table: (transaction_id, item_id)

3. First Normal Form (1NF) Conversion

Explanation: 1NF mandates atomic values and no repeating groups. The current table already aligns with 1NF since every cell holds a discrete value and each row corresponds to a single item per transaction.

```
-- Display 1NF records
SELECT transaction_id, transaction_date, buyer_id, buyer_name,
       agent_id, agent_name, item_id, item_name, qty
FROM Retail_Sales
ORDER BY transaction_id, item_id;
```



4. Second Normal Form (2NF) Conversion

Explanation: 2NF eliminates partial dependencies. Currently, non-key attributes depend on parts of the composite key (e.g., item details depend only on item_id). We resolve this by splitting into Transaction_Header, Item_Catalog, and Transaction_Details.

```
-- Create transaction header table
CREATE TABLE Transaction_Header AS
SELECT DISTINCT transaction_id, transaction_date, buyer_id,
buyer_name, agent_id, agent_name
FROM Retail_Sales;

-- Create item catalog table
CREATE TABLE Item_Catalog AS
SELECT DISTINCT item_id, item_name
FROM Retail_Sales;

-- Create transaction details table
CREATE TABLE Transaction_Details AS
SELECT transaction_id, item_id, qty
FROM Retail_Sales;

-- Display normalized tables
SELECT * FROM Transaction_Header ORDER BY transaction_id;
SELECT * FROM Item_Catalog ORDER BY item_id;
SELECT * FROM Transaction_Details ORDER BY transaction_id,
item_id;
```



5. Third Normal Form (3NF) Conversion

Explanation: 3NF targets transitive dependencies. In our header table, buyer_name relies on buyer_id, and agent_name relies on agent_id (non-key attributes determining other non-key attributes). We branch these into separate entities.

```
-- Create buyer table
CREATE TABLE Buyer AS
SELECT DISTINCT buyer_id, buyer_name
FROM Transaction_Header;

-- Create agent table
CREATE TABLE Agent AS
SELECT DISTINCT agent_id, agent_name
FROM Transaction_Header;

-- Create normalized transaction header
CREATE TABLE Transaction_Header_3NF AS
SELECT transaction_id, transaction_date, buyer_id, agent_id
FROM Transaction_Header;

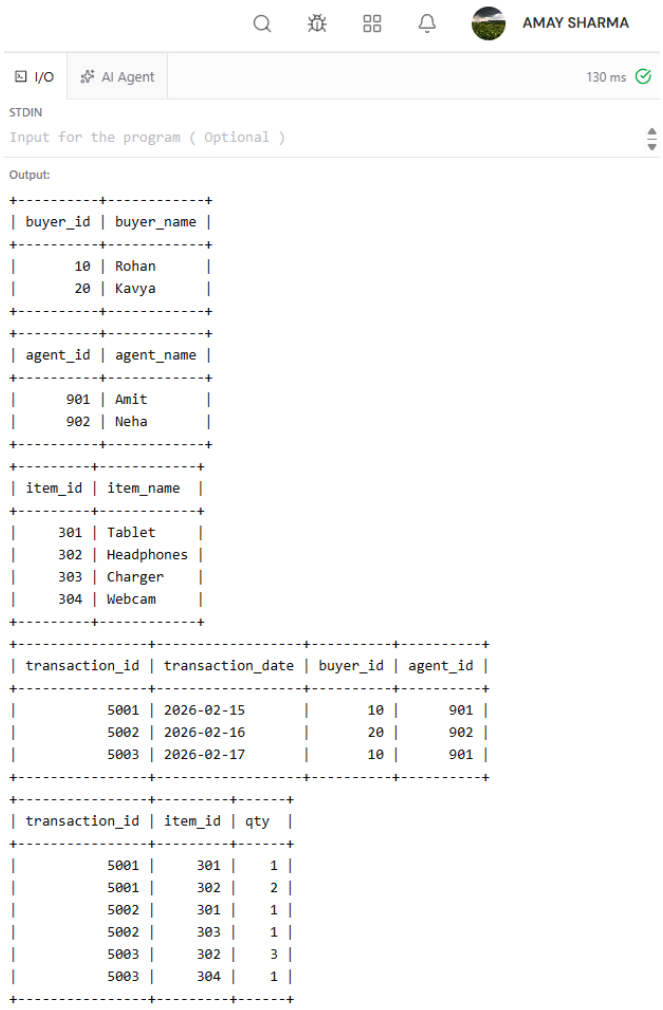
-- Display 3NF tables
SELECT * FROM Buyer ORDER BY buyer_id;
SELECT * FROM Agent ORDER BY agent_id;
SELECT * FROM Transaction_Header_3NF ORDER BY transaction_id;
```



6. Boyce-Codd Normal Form (BCNF) Verification

Explanation: BCNF dictates that for any dependency A → B, A must be a superkey. Our decomposed tables conform to BCNF because all determinants (buyer_id, agent_id, item_id, transaction_id) act as primary/candidate keys in their respective new tables.

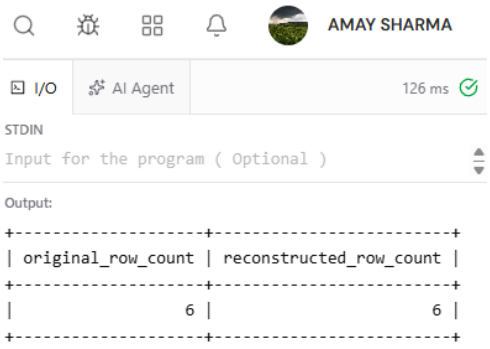
```
-- Verify records in BCNF tables
SELECT buyer_id, buyer_name FROM Buyer;
SELECT agent_id, agent_name FROM Agent;
SELECT item_id, item_name FROM Item_Catalog;
SELECT transaction_id, transaction_date, buyer_id, agent_id FROM
Transaction_Header_3NF;
SELECT transaction_id, item_id, qty FROM Transaction_Details;
```



7. Verify Lossless Join

Explanation: A proper decomposition allows us to recreate the original dataset perfectly using JOIN operations, ensuring no ghost rows appear and no actual records are lost.

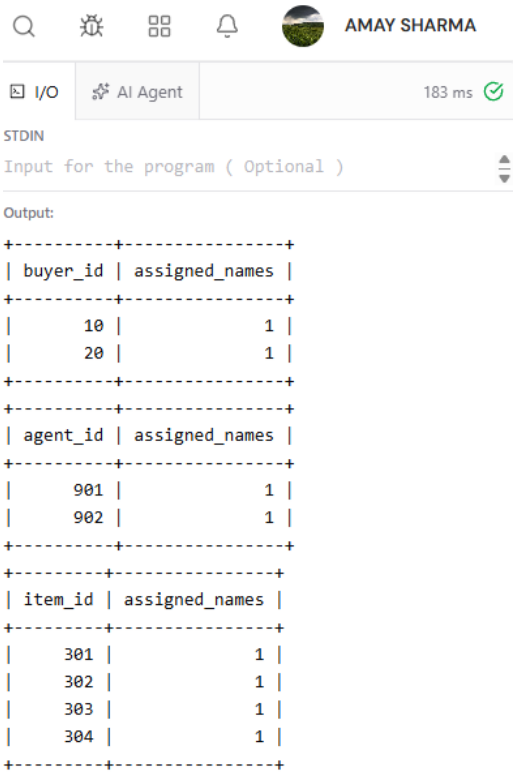
```
-- Compare original and reconstructed row counts
SELECT
    (SELECT COUNT(*) FROM Retail_Sales) AS original_row_count,
    (
        SELECT COUNT(*)
        FROM Transaction_Header_3NF H
        JOIN Buyer B ON H.buyer_id = B.buyer_id
        JOIN Agent A ON H.agent_id = A.agent_id
        JOIN Transaction_Details D ON H.transaction_id =
D.transaction_id
        JOIN Item_Catalog I ON D.item_id = I.item_id
    ) AS reconstructed_row_count;
```



8. Verify Dependency Preservation

Explanation: This confirms that our primary functional dependencies can still be validated locally within the new individual tables, avoiding complex cross-table joins for constraint checking.

```
-- Check individual dependencies
SELECT buyer_id, COUNT(DISTINCT buyer_name) AS assigned_names
FROM Buyer GROUP BY buyer_id;
SELECT agent_id, COUNT(DISTINCT agent_name) AS assigned_names
FROM Agent GROUP BY agent_id;
SELECT item_id, COUNT(DISTINCT item_name) AS assigned_names
FROM Item_Catalog GROUP BY item_id;
```



RESULT

The database was successfully transitioned from a denormalized state to BCNF. Dependency preservation and lossless joins were empirically verified through SQL queries, returning matching row counts and single-name mappings.

CONCLUSION

Through progressive normalization, we eradicated redundant data blocks, safeguarding the system from insertion, update, and deletion anomalies while maintaining structural integrity.